DATABRICKS · AI ENGINEERING

3 Ways to Build AI Agents in Databricks

One banking dataset. One tool layer. Three approaches — from no-code to full production.

1 · AI Playground

2 · Agent Bricks

3 · Mosaic AI Agent Framework

THE PLAN

The same agent, built three ways

One agent. Three questions it can answer:

Look up a customer's account balance

Explain why a card transaction was declined

Calculate credit utilisation & available credit

Same data · same tools · different amounts of code

- **No-code**
  AI Playground
- **Low-code**
  Agent Bricks
- **Full-code**
  Mosaic AI Agent Framework

Everything is on GitHub — link in the description.

BEFORE ANY AGENT, WE NEED DATA

Meet ABCBank — a synthetic bank

6

synthetic datasets

5,000

transactions

including declines with reasons

1. **customers**
2. **accounts**
3. **deposit accounts**
4. **credit-card accounts**
5. **transactions**
6. **support tickets**

Generated in Python with Faker. **No real personal or financial data anywhere** — clone the repo and run it safely.

SETUP · RUN ONCE

Catalog, schemas, tables

Two notebooks in uc_setup/ get your lakehouse ready:

01

Create the structure

One catalog and two schemas: core for the data, tools for the functions — plus the six tables.

02

Generate the data

02_generate_sample_data.py builds the rows from the data-generation scripts. Plain Python — read it in the repo.

`core` data tables `tools` UC functions

THE TOOL LAYER

Tools = Unity Catalog functions

Write the COMMENT for the model. It's not decoration — the LLM reads the function's description to decide when to call it.

`get_customer_balance()`

Current balance for a customer.

`calculate_credit_utilization()`

Utilisation % and available credit.

`get_recent_transactions()`

A customer's latest transactions.

`check_transaction_status()`

Status — plus a plain-English reason a transaction was declined.

Smoke-test each one before handing them to an agent — e.g. check_transaction_status('TXN00000010').

APPROACH 1 · NO-CODE

AI Playground — where you experiment

1. Pick a tool-calling model, click Tools, add the 4 UC functions.
2. Paste a system prompt: never guess numbers, chain tools when needed.
3. Ask, watch the trace, swap the model, re-run to compare.

Best for fast experiments with prompts, tools, and models.
Not for production.

▸ Scenario 4 (tools chained automatically)

"Why was TXN00000010 declined, and what's my available credit?"

```
1  check_transaction_status
→ insufficient credit limit

2  calculate_credit_utilization
→ 86.9% · ₹39,263 available

✓  combined answer + next step
one reply, two tools
```

APPROACH 2 · LOW-CODE

Agent Bricks — auto-optimised patterns

Describe the task, connect your data — Agent Bricks generates evals, uses an LLM judge, and tunes the prompt, retrieval, tools, and model **automatically.**

Supervisor

Orchestrate multiple agents & tools into one workflow.

Information Extraction ← *our pick*

Turn documents (PDFs, tickets, reports) into structured fields.

Genie

Natural-language questions over your governed tables.

Text Classification

Sort text into your own categories at scale.

Custom

Custom text transformation & generation tasks.

The pick: Information Extraction — auto-optimised over ABCBank's support tickets, with **one-click serving, a review app, and built-in traces** — no code written.

APPROACH 3 · FULL-CODE

Mosaic AI Agent Framework — full control

Build on MLflow's `ResponsesAgent` interface — you get Playground compatibility, evaluation, tracing, and deployment for free.

`model_config.py`

Config in one place

Catalog, LLM endpoint, MCP path, system prompt, tool-iteration cap.

`tools.py`

The MCP bridge

Lists MCP tools, converts them for the LLM, routes calls back. Scopes exact permissions.

`agent.py`

The predict loop

Send messages + tools → run tool → feed result → loop to cap → return answer + tools used.

`@mlflow.trace` traces every step. Then one call — `agents.deploy` — ships a governed, monitored endpoint.